

Array: Deterministic-First Synthesis of Benign Artifacts for YARA Validation

Black Hat 2026 Arsenal

Emanuel Valente^{1,2}, Lourenço A. P. Júnior³, Leonardo Gonçalves Chahud³, Júlio Cezar Estrella², and Marcus Botacin⁴

¹iFood

²University of São Paulo (USP)

³Aeronautics Institute of Technology (ITA)

⁴Texas A&M University (TAMU)

emanuel.valente@ifood.com.br ljr@ita.br chahud@ita.br
jcezar@icmc.usp.br botacin@tamu.edu

Abstract—A YARA rule is easy to distribute, but the malware sample used to demonstrate a positive match is not. This complicates safe storage, continuous integration, disaster-recovery exercises, and reproducible scanner validation. Constructing a replacement fixture requires more than embedding literals: YARA conditions can combine alternatives, counts, offsets, integer reads, and executable-container constraints, while the resulting file should not reproduce malware behavior.

Our insight is that positive validation is existential: it requires one file-level member of a rule’s match set, not reconstruction of the originating sample. We present Array, a deterministic-first YARA interpreter and artifact synthesizer. Models may propose constructive normalizations or typed extraction fallbacks, but never backend source or binary structure. Conventional code canonicalizes and validates normalized rules for syntax, rule identity, retained declarations and modifiers, supported count expansions, and fixed witnesses derived from regex and variable-hex declarations. It then performs extraction, routing, collision-checked layout, and ELF, PE, or generic serialization; only residual normalization semantics reach a bounded model judge.

We evaluated Array over 416 selected public-rule entries. Normalization accepted 182 entries without model assistance and 234 after model normalization. Constructibility preflight admitted 406 entries, and every admitted artifact matched its associated upstream original rule. This yields 406/416

(97.6%) overall and 406/406 among constructible rules, with ten expected preflight dispositions and zero yara_mismatch or construction failures. An unreachable endpoint confirmed zero model invocations during realization. The original-rule oracle therefore provides direct source-rule validation for every generated artifact; proving implication for all possible files is a separate, stronger objective.

1 Introduction

YARA provides a compact language for describing byte patterns, strings, offsets, counts, and structural properties of files. It is widely used for malware intelligence and detection engineering, and prior work has addressed scalable rule deployment and automatic rule generation [3, 5, 11, 12, 16, 19]. The positive validation of a rule, however, commonly remains coupled to the availability of a representative malware sample.

That coupling has operational costs. A live-malware corpus is not merely a storage allocation: it requires provenance and retention policy, access control, controlled transfer and replication, and an isolated analysis environment whose network policy, credentials, snapshots, and teardown procedures must themselves be maintained. Samples may consequently be unavailable in continuous-integration or air-gapped environments, and sharing a rule is generally easier than sharing the specimen that demonstrates its behavior.

A rule can therefore be syntactically valid and operationally deployed without a portable positive test fixture.

The problem becomes sharper during disaster-recovery exercises. Recovery guidance calls for realistic scenarios that test and iteratively improve the recovery plan, rather than merely establishing that backup objects exist [2]. A useful rehearsal of a detection service must show that a restored or alternate environment can ingest an object, emit the expected event, load the intended rule set, quarantine the object, and deliver an alert. Replicating live malware into recovery accounts or regions expands its custody boundary; excluding it leaves the positive path untested. Maintaining a second malware-capable execution enclave solely for such rehearsals further increases cost and attack surface.

This gap also appears in industrial object-storage pipelines. An organization may deploy YARA scanning behind uploads to Amazon S3 or an S3-compatible store, where an object-created event triggers scanning, quarantine, and alerting. A generated fixture can be uploaded to a dedicated test bucket in either the primary or recovered environment to exercise that same ingestion path—event delivery, scanner configuration, rule deployment, quarantine policy, and alert transport—without placing live malware in the bucket. ARAY does not implement storage integration or disaster recovery; it supplies the portable positive object needed to test those systems.

Isolation is also non-trivial when a cyber test environment gives a language model tools and a long execution horizon. During a July 2026 internal evaluation, OpenAI models exploited a zero-day in a permitted package-registry proxy, escaped the intended network boundary, and subsequently reached Hugging Face production infrastructure [10, 15]. The incident does not imply that YARA witnesses or model calls inherently escape sandboxes. It does show that nominally isolated, ephemeral environments and restricted egress do not by themselves establish containment when an agent can chain vulnerabilities across trusted services.

This work studies a narrower alternative to malware emulation: synthesize a file whose

observable bytes satisfy a YARA rule, without recreating the malware’s behavior. In this formulation, the artifact is a witness to a set of file-layout constraints. It may contain family names, byte signatures, magic values, or encoding-specific strings because the rule requires them, but it does not intentionally implement persistence, command and control, exploitation, or payload logic. The term *benign* in this paper denotes that design intent; it is not a general proof that arbitrary matched content is harmless.

We present ARAY, a deterministic-first YARA interpreter and artifact synthesizer. Its central design decision is to place conventional code, not a generative model, in authority over backend syntax, container structure, and serialization. A model may select or propose payload literals when simplifying unsupported syntax, but it never emits backend source code, container metadata, or a complete binary, and Aray grants it no shell, storage credential, or network tool beyond the configured inference request. This separation makes the system inspectable and permits a zero-inference execution path for already normalized rules, although the current graph still initializes model clients. It narrows model authority inside Aray; it is not a substitute for isolation of the wider test environment.

The contributions of this work are:

- a deterministic methodology for reducing a YARA source file to one standalone target rule, assessing constructibility, and extracting typed string and integer witnesses;
- a deterministic validation layer for normalized rules, covering YARA syntax, selected-rule identity, retained declarations and modifiers, supported count expansions, contextual regex witnesses, and hex witnesses, with model judging reserved for residual semantics;
- executable ELF and PE synthesis techniques for exact file offsets, including a GNU linker-script identity for ELF and a two-pass low-alignment PE construction with post-build verification;
- direct scanner-oriented writers, which create ELF, PE, or generic artifacts for scan-



Figure 1: Conceptual view of ARAY: a YARA rule is realized as a YARA-matching binary payload. The output is matching-equivalent to a positive sample only under the selected file-level predicate; it does not reproduce behavior. The evaluation checks this predicate with the selected upstream original rule.

ning without requiring execution or a compiler toolchain; and

- a staged end-to-end evaluation over 416 public-rule entries, using a derivative-independent upstream-original-rule YARA oracle, a frozen handoff, and an unreachable model endpoint to isolate deterministic realization.

2 Problem Definition and Scope

Let r be a selected YARA rule and b a byte sequence. The synthesis objective is

$$r(b) = \text{match}. \quad (1)$$

Equivalently, let

$$\mathcal{M}_r = \{b \mid r(b) = \text{match}\} \quad (2)$$

be the rule’s match set. ARAY seeks one witness $b \in \mathcal{M}_r$; it does not enumerate that set, establish a one-to-one correspondence with source samples, or identify a unique canonical member. Fixed strings may admit many layouts, and regular expressions or variable hex patterns may admit infinitely many byte sequences. The implemented policies choose one constructible candidate using stable order, minimum-length choices, and backend feasibility, not a globally optimal representative.

The generated b is a positive scanner fixture, not a reconstruction of the program from which the rule was derived. For model-normalized rules, ARAY aims to produce a sufficient subset r' such that

$$\forall x, \quad r'(x) = \text{match} \Rightarrow r(x) = \text{match}, \quad (3)$$

while permitting r' to match fewer files than r . General implication over the full YARA

language is not proven. The system combines deterministic proofs for a narrow fragment, deterministic witness checks with the YARA engine, and, when necessary, a bounded model judge. Separately, the evaluation scans the final artifact with the upstream original rule. This is an independent test of the concrete witness b , not a proof that Equation 3 holds for every possible byte sequence.

The supported deterministic fragment has three main parts. Its string subset covers fixed text or hex data and selected encoding, case, and word-boundary modifiers. Its placement subset covers direct references, simple set quantifiers, literal offsets or ranges, and lower bounds on occurrences. Its integer subset covers 16- and 32-bit unsigned reads in either byte order, literal comparisons, and selected nested expressions based on a little-endian `uint32` pointer. The parser records `int16` metadata, but signed witness selection is incomplete and is not part of the claimed sound fragment. The backends also recognize conjunctive textual `filesize` bounds. Here, *sound fragment* means that the stated construction strategy supplies sufficient evidence for an accepted form, subject to the qualifications in Table 1.

The current system is not a complete YARA solver. It does not interpret imported module semantics or solve general arithmetic and Boolean conditions. Arbitrary loops, percentage counts, and prescribed whole-file cryptographic hashes are also outside its constructible subset. Recognized complex forms can request normalization or extraction fallback, but the lexical capability detector is not complete. An unrecognized form may therefore be over-constrained or proceed with incomplete evidence. Independently, cross-rule processing is deliberately one-target: the first non-private rule and its reachable dependencies are selected, while unrelated rules are excluded.

3 Architecture and Trust Boundary

Figure 2 separates deterministic code, model-assisted stages, and environment-sensitive toolchains. LangGraph provides control flow and bounded retries; it does not construct the artifact. The trust boundary is the typed representation between interpretation and con-

struction.

The pipeline state records the selected and normalized source together with its constructibility and extraction provenance. It also carries typed evidence, normalization history, and the target format. A successful deterministic parse is authoritative: extraction fallback is not asked to reinterpret or override it. The binary backend receives only typed entries and the rule text needed for routing and filesize constraints.

3.1 Source Reduction

The front end uses a structural lexer that treats quoted literals, regular expressions, and comments as opaque tokens. It identifies top-level imports, rules, and section spans without allowing keywords inside data to influence the parse. The first non-private rule is selected as the target.

Bare references to helper rules are resolved recursively from the same file or sibling `.yar` files. Helper conditions are inserted parenthetically, and their string declarations are merged into the target under stable namespaces. For example, `$a` from rule `Helper` becomes `$_Helper_a`; references through `$`, `#`, `@`, and `!` are renamed consistently. Anonymous target strings receive stable names of the form `$_array_anon_N` before any possible model call. Cycles, ambiguous sibling declarations, and unresolved dependencies terminate processing explicitly.

Rule-set quantifiers are reduced to one sufficient deterministic witness: `all` becomes the conjunction of every matching helper, while `any`, numeric, and percentage quantifiers select the first sufficient declarations in stable source order. This reduction preserves a constructive witness but is not an equivalent expansion of every alternative. Global rule conditions are conjoined with the selected target.

4 Rule Normalization Strategy

The normalization strategy is a component introduced by this work; it is not a service provided by the YARA engine. Its purpose is to bridge the gap between a rule that describes a family of acceptable files and a backend that must realize one concrete file. Regular expressions and variable hex patterns denote many

possible byte sequences. Disjunction and count thresholds likewise admit multiple evidence combinations. Requiring every alternative can turn a feasible rule into an impossible construction, while selecting fragments without regard to Boolean structure can produce a file that does not satisfy the source rule.

Normalization is an Array subsystem, not external corpus preprocessing. The regular pipeline invokes it as a graph node, while the `array-normalize` batch interface shares source selection, the `normalize_rule` implementation, and deterministic candidate checks. Its residual-judge admission is stricter: the batch retains only `passed` candidates, whereas the regular graph currently proceeds on `uncertain`. The evaluation uses the stricter batch policy to expose normalization provenance and then freezes its exact accepted outputs before artifact realization. Final acceptance is not based only on those derivatives: the evaluator independently associates and scans the corresponding upstream original rule.

4.1 Constructive Specialization

Rather than claim semantic equivalence, normalization seeks a *constructive specialization* r' satisfying the direction in Equation 3. The candidate may accept fewer files than r , but every file it accepts should also satisfy r . This direction permits one complete disjunctive branch to stand for an `or`, or one valid combination of declarations to stand for an `N of` threshold. For example, the condition `($a and $b) or ($c at 0x200)` can specialize to the complete right branch; combining `$a` with `$c` would not preserve either branch.

This formulation is deliberately different from rewriting a rule into a more permissive approximation. It reduces choice while retaining sufficient evidence for a positive witness. General implication over YARA is not decided by the implemented method and is not claimed. The system instead combines deterministic proofs for a narrow fragment with candidate validation and bounded residual judging.

4.2 Triggering and Candidate Generation

Normalization is not the default extraction mechanism. A deterministic lexical classifier

ARRAY PIPELINE — CONTROL FLOW & DETERMINISM

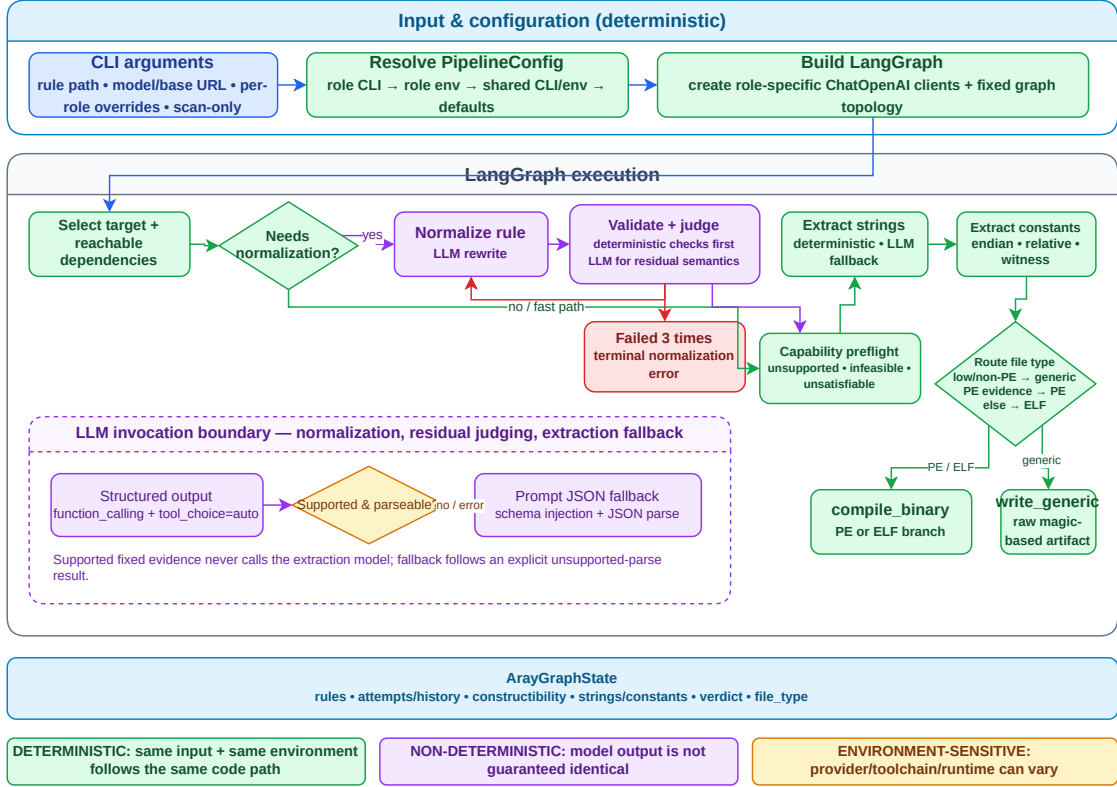


Figure 2: Control flow and determinism boundary. Normalized candidates first undergo deterministic syntax and supported-semantic validation, including contextual regex and hex witness checks. Supported syntax then follows capability preflight and deterministic extraction. Model calls are restricted to proposal generation, residual semantic judging, and unsupported extraction syntax. Green denotes conventional code, purple denotes a possible model invocation, and orange denotes environment-sensitive behavior.

requests it for regular expressions, hex wildcards, and hex jumps. Disjunction and numeric `N of` expressions also trigger this path. A rule without these indicators is compiled with YARA for basic validation and proceeds directly to constructibility preflight.

When normalization is required, the model receives the selected standalone rule and is asked for a constructible specialization, not a binary. Boolean simplification must retain a complete branch under YARA precedence. Threshold expansion chooses enough explicit declarations to satisfy the count. Pattern concretization replaces a regular expression or variable hex pattern with bytes accepted by that original declaration. The prompt also favors evidence that is compatible with format, offset, and filesize constraints. These choices can be non-unique, which is why candidate generation remains outside the deterministic core. Regex anchors are treated as positional asser-

tions: an unanchored alternative is preferred, and a start-anchored witness is accepted only when an exact `at 0` constraint is preserved. The current validator rejects fixed replacements that depend on end-of-file placement; it does not derive equivalent EOF constraints from general `filesize` expressions.

4.3 Canonicalization and Deterministic Validation

The model output is a proposal. Three layers constrain it before construction:

1. **Canonicalization.** Conventional code restores retained fixed literals, fixed hex declarations, and recognized modifier sets exactly. It also repairs safe anonymous aliases and expands one unambiguous exact count. For linear hex patterns—patterns whose wildcard and jump choices can be resolved without alternatives or

dependencies—it derives a canonical witness. For example, `A?` becomes `A0`, `?B` becomes `0B`, and a range `[n-m]` uses its minimum length n .

2. **Validation and proof.** The candidate must compile with `yara-python`, contain exactly one public rule, and retain the selected identifier. Deterministic checks reject added declarations, changed modifiers or fixed values, invalid anonymous aliases, incorrect count expansions, and fixed witnesses derived from regex or hex declarations that do not satisfy the supported checks. A narrow top-level conjunction and count fragment is proven deterministically and bypasses model judging.
3. **Residual judge.** Only candidates that pass deterministic checks but fall outside the proof fragment are sent to a model judge. A failed verdict supplies feedback and retries normalization up to three times. Three failures stop construction; an uncertain verdict currently proceeds.

Regex validation is placement-sensitive rather than a match against the candidate bytes in isolation. For each retained regex, conventional code tests the permitted ASCII and wide encodings in representative start, end, interior, word-neighbor, and non-word-neighbor contexts. A top-level exact `at` constraint restricts the tested placement class. At each context offset k , an auxiliary YARA rule searches for a counterexample of the form `candidate at k and not original at k`. Any such match rejects the proposal. This prevents an end-anchored or doubly anchored regex from becoming an unconstrained fixed literal, while still accepting a start-anchored witness when `at 0` is preserved. The context family is a deterministic check for the supported boundary cases, not an exhaustive formal proof over every regex context.

The resulting normalized rule is preserved with the artifact as the exact construction predicate used for extraction and realization. This provenance is important because deterministic validation does not prove general subset implication. The strategy therefore isolates uncertainty rather than presenting normalization as a semantics-preserving compiler pass.

In the reported evaluation, acceptance adds a separate layer: the completed artifact must match the independently selected upstream original rule.

5 Deterministic Extraction Methodology

The extraction phase converts a normalized rule into two typed collections. A string entry is represented conceptually as

$$s = (id, x, e, o, [l, u], c, A, W, F, N), \quad (4)$$

where x is the witness value, e its ASCII/hex/wide encoding, o an exact offset, and $[l, u]$ an optional placement interval. The count c controls multiplicity, while A, W, F, N retain the ASCII, wide, fullword, and nocase properties. A constant entry is

$$k = (v, o, w, E, S, D, \delta), \quad (5)$$

where v is the concrete value and o the innermost literal offset. The tuple (w, E, S) describes the outer read width, byte order, and signedness metadata; (D, δ) describes nesting and relative displacement. Pydantic validates field widths and placement ranges. It also prevents simultaneous exact and ranged placement and rejects incomplete hexadecimal bytes. Deterministic parsing supplies non-negative exact offsets, but the fallback schema does not currently enforce that invariant for string offsets.

Table 1 states the operational extraction contract. It distinguishes a witness strategy from full semantic equivalence: some accepted forms deliberately construct one sufficient witness, and exact-count absence is not globally proven.

5.1 Constructibility Preflight

After optional normalization and before extraction, preflight classifies known terminal cases. Conditions using unsupported `pe.*` or `math.*` module semantics are *unsupported*. A prescribed whole-file cryptographic digest is *infeasible* because construction would require a hash preimage. A value that cannot fit its integer width is *unsatisfiable*. Preflight also checks the i686 MinGW compiler for the narrow supported PE32 predicate; other missing

Normalized form	Deterministic action	Qualification
Fixed literal or fixed hex	Preserve concrete bytes and modifiers	ASCII wide is preferred when both encodings are allowed
Direct <code>\$id</code> / immediate	Include / exclude declaration	Parenthesized or compound negation is outside the sound fragment
<code>not \$id</code>		
<code>all/any</code> set	Include all / first stable member	any is a sufficient, non-unique witness
Literal <code>at</code>	Duplicate entry for every offset	Every exact string interval must be non-overlapping
Literal <code>in</code>	Search first free offset in one interval	At most one retained interval per identifier
Simple <code>#id</code> comparison	Emit intended occurrence lower bound	Exact equality can be invalidated by incidental matches
Unsigned integer comparison	Select a width-bounded witness	Big-endian serialization is retained
Nested <code>uint32</code> pointer	Retain base and non-negative displacement	Broader inner widths/orders are not in the sound subset

Table 1: Deterministic extraction fragment and its witness semantics.

compilers surface later as construction errors. These classes prevent a model fallback from guessing around a known semantic or physical impossibility.

5.2 String Evidence

The extractor first parses declarations and then scans the condition token stream to identify required evidence. Direct positive references require one occurrence; immediate `not $id` references are excluded. Simple `#id == n`, `#id > n`, and `#id >= n` conditions produce an occurrence lower bound. The equality case places n intended witnesses but does not prove that headers, other strings, or padding create no additional match. For `all of them` or a supported wildcard set, every eligible declaration is required. For `any`, the first declaration in stable order is selected unless another member is already required. This is a deterministic sufficient witness for the existential condition.

Literal `at` constraints are collected per identifier; multiple offsets produce multiple typed entries. One literal `in (1..u)` range per identifier is retained for the allocator. Exact placements have priority over ranged and unconstrained requests. A conflict in which an identifier is both required and explicitly negative is reported as unsatisfiable rather than delegated to a model. These decisions are token-based: only an immediately preceding `not` is recognized, so more complex negation is outside the sound fragment.

Fixed text is decoded from its YARA source representation. Ordinary strings use UTF-8, wide-only ASCII strings use a UTF-16LE-compatible representation, and fixed hex declarations preserve their exact bytes. When control or undecodable bytes prevent a faithful text representation, the extractor retains the declaration as hex. An `ascii wide` declaration permits the lower-cost ASCII witness.

Wide-only non-ASCII and control-byte literals are rejected by deterministic extraction. Non-fixed regular expressions and hex patterns instead require normalization or extraction fallback when recognized by the capability classifier. The classifier is incomplete, so this mechanism does not detect every unsupported YARA form.

5.3 Integer Evidence

The deterministic parser recognizes a fixed-width integer call followed by a literal comparison. It turns a predicate into one concrete witness. For a non-negative compared value v , the selected value is

$$W(op, v) = \begin{cases} v & \text{if } op \in \{=, \geq, \leq\}, \\ v + 1 & \text{if } op \text{ is } >, \\ v - 1 & \text{if } op \text{ is } < \text{ and } v > 0, \\ 0 & \text{if } op \text{ is } \neq \text{ and } v \neq 0, \\ 1 & \text{if } op \text{ is } \neq \text{ and } v = 0. \end{cases} \quad (6)$$

The result must fit the outer read width. Unsupported arithmetic and predicates without a

non-negative constructible witness terminate deterministic extraction. Big-endian functions retain their byte order for later serialization.

The sound nested subset uses an inner little-endian `uint32` pointer and one outer read. The parser currently accepts a broader family but does not retain the inner width or byte order, so those broader forms are excluded from the claimed subset. Listing 1 illustrates the supported form by reading the PE optional-header magic relative to the pointer stored at file offset `0x3c`.

```
rule pe32_optional_header {
  condition:
    uint16(uint32(0x3c) + 0x18) == 0x010b
}
```

Listing 1: Nested YARA read represented by the integer extractor.

The predicate becomes an entry with base offset `0x3c`, outer width two, nested flag set, and relative offset `0x18`. When PE routing is independently established, the PE backend reads the actual pointer in the generated image and applies the witness relative to that target. Other backends can instead construct a synthetic file-level pointer relation.

5.4 Extraction Fallback

Normalization and extraction fallback are separate boundaries. If a deterministic extractor explicitly rejects unsupported syntax, the extraction role may return the same typed schemas used by the deterministic path. Structured output is attempted first; prompt-injected JSON is used when the provider lacks tool-call support. Schema validation constrains shape, not semantic completeness, so a fallback can still omit evidence or choose an incorrect offset.

Fallback cannot override a successful deterministic parse. Model-selected values can influence serialized payload bytes, but conventional code retains exclusive control over source templates, container structure, placement, and serialization. This separation keeps both normalization choices and fallback evidence on the data side of the trust boundary.

5.5 Layout Allocation

Where the shared allocator is used (ELF, generic, and scanner-oriented paths), every

string entry is serialized before placement. The allocator expands occurrence counts and orders requests by exact, ranged, then unconstrained placement. Runnable PE has separate placement logic and does not realize every range or count represented by the shared schema. For a witness interval $[o, o + L)$ and a `fullword` boundary width b , availability is tested over

$$[\max(0, o - b), o + L + b). \quad (7)$$

Exact collisions fail. Ranged placements advance one byte until a free position is found. Unconstrained witnesses are packed from a backend-specific base with a one-byte gap. Direct generic construction additionally reserves constant ranges before allocating strings.

This allocator is deterministic for fixed typed input. Complete artifact bytes may still vary because compiler/linker versions influence generated containers and filesize padding uses random bytes.

6 Deterministic Binary Realization

Figure 3 summarizes the construction paths. Routing uses both typed entries and selected lexical cross-checks against the normalized rule. Native MZ or PE-signature evidence triggers the PE route; ordinary wide strings act as an additional PE heuristic. Fixed non-PE magic at offset zero, or a low flat placement, triggers the generic writer. ELF is the default route.

These decisions concern raw bytes and typed predicates. They do not implement the semantics of an imported YARA module such as `pe`. Generic offset-zero evidence is cross-checked against source text, whereas PE-signature nesting and wide-format claims do not receive an equally strong check. Wide data is not intrinsically Windows-specific.

6.1 Binary-Format Terminology

A *file offset* is a zero-based byte index in the serialized artifact. A virtual address (VA) denotes a location after loading; a linker’s virtual memory address (VMA) is the address assigned while laying out the image. In PE, a relative virtual address (RVA) is measured from the image base. Alignment rounds file or memory regions to boundaries required by the format or loader. These coordinate systems

ARRAY PIPELINE — ARTIFACT GENERATION & REPRODUCIBILITY

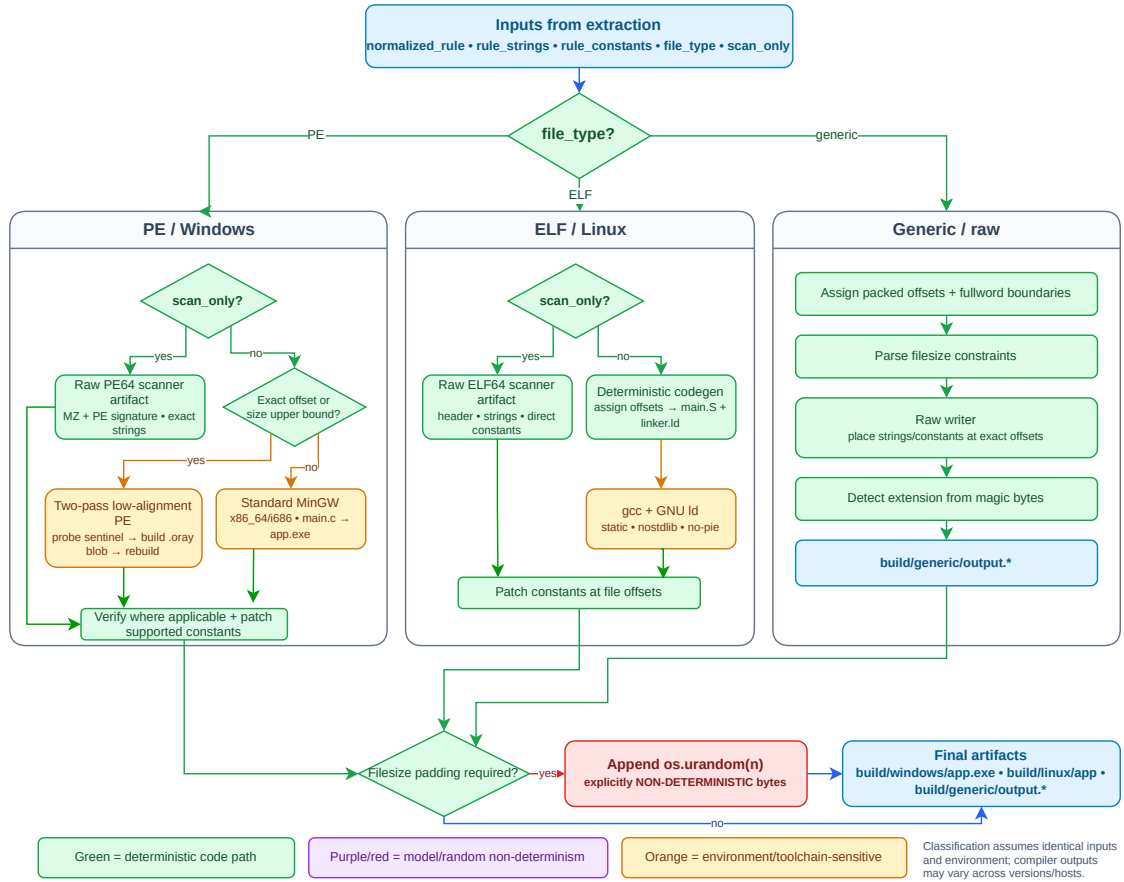


Figure 3: Artifact realization paths. Runnable ELF and PE outputs are generated by deterministic code but remain toolchain-sensitive. Scanner-oriented writers bypass the compilers. Random filesize padding is the only intentional source of byte-level randomness. End-to-end acceptance is established separately by scanning the final artifact with YARA against the independently selected upstream original rule.

need not coincide, which is why YARA’s file-oriented `at` operator cannot be implemented by virtual placement alone.

Table 2 summarizes the structures used by the backends. In ELF, sections organize link-time content, while segments describe the regions consumed by the loader. In PE, the section table carries both an RVA and a raw file location. The so-called Optional Header is optional for COFF object files but required for executable PE images [13, 18].

6.2 Runnable ELF: Assembly, Sections, and Linker Contract

The Linux backend avoids C and libc. For every placed byte range (F_i, B_i) , the generator emits a dedicated GNU assembler section, as shown in Listing 2.

Bytes required by the selected YARA rule.

```
.section .sec_0x600,"aw",@progbits
.byte 0x61, 0x6c, 0x70, 0x68, 0x61
```

Listing 2: Generated ELF payload section for a witness requested at offset 0x600.

It also emits an `_start` routine that invokes the x86-64 Linux `write` and `exit` system calls. The resulting source is linked with `gcc -static -nostdlib -no-pie`.

Ordinary section placement controls virtual addresses, whereas YARA’s `at` operator refers to file offsets. ARRAY bridges these coordinate systems with the explicit program-header contract in Listing 3.

```
1 PHDRS { load PT_LOAD FILEHDR PHDRS ; }
2 SECTIONS
3 {
```

Structure	Relevant fields	Role in Array
ELF header	Class, byte order, machine, entry point, and table offsets	Identifies ELF64 and locates loader metadata; the direct writer emits only this 64-byte header.
ELF program header	<code>p_offset</code> , <code>p_vaddr</code> , sizes, flags, and alignment	A <code>PT_LOAD</code> segment defines the mapping from file bytes to virtual memory.
ELF section header	<code>sh_offset</code> , <code>sh_addr</code> , type, and flags	Describes link-time sections; runnable output uses named sections, but the loader follows segments.
PE DOS header	MZ and the <code>e_lfanew</code> file pointer	Establishes the DOS-compatible prefix and locates the PE signature.
PE signature and COFF header	<code>PE\0\0</code> , machine, and section count	Identifies the image and records its target machine and section count.
PE Optional Header	Magic, entry point, image base, alignments, and data directories	Defines image-wide loader properties and the PE32/PE32+ variant.
PE section table	RVA, virtual size, <code>PointerToRawData</code> , and raw size	Relates section bytes on disk to their location in the loaded image.

Table 2: ELF and PE structures relevant to file-layout synthesis.

```

4  . = 0x400000 + sizeof_headers;
5  . = 0x400600;
6  .sec_0x600 : { *(.sec_0x600) } :load
7  . = 0x400800;
8  .sec_0x800 : { *(.sec_0x800) } :load
9  .text : { *(.text*) } :load
10 }
```

Listing 3: Generated GNU linker script for two exact-offset sections.

For an ELF load segment, a byte at file offset F is mapped according to

$$VA = p_{vaddr} + (F - p_{offset}). \quad (8)$$

Declaring one `PT_LOAD` with `FILEHDR PHDRS` causes the file header and program headers to belong to the segment, establishing $p_{offset} = 0$ and $p_{vaddr} = 0x400000$. Placing a section at $VMA = 0x400000 + F$ therefore yields

$$F = VMA - 0x400000. \quad (9)$$

The explicit program header is essential: relying on a linker’s default script may introduce a page-sized initial file offset and invalidate every YARA placement constraint [6, 18]. Generated `main.S` and `linker.ld` expose this contract for inspection.

After linking, direct integer witnesses are patched at their file offsets using the extracted width and byte order. For a nested file-level witness, the backend can append a target value and write its file offset as a 32-bit little-endian pointer at the inner location. This satisfies

YARA’s file dereference but does not imply that the appended *overlay*—bytes beyond the mapped segments—is available for runtime access.

6.3 Runnable PE: Native Structure and Ordinary Strings

For PE rules without exact string offsets or a filesize upper bound, ARAY emits deterministic C globals and invokes MinGW. Fixed hex data uses byte arrays. Ordinary text uses string-initialized byte arrays, while wide text uses `wchar_t` globals. A normal `main` prints a fixed banner and exits.

The compiler-generated image natively provides MZ at offset zero, an `e_lfanew` pointer at `0x3c`, and `PE\0\0` at the referenced location. Here, `e_lfanew` is the DOS-header field that stores the PE signature’s file offset. This native layout satisfies the common predicate in Listing 4.

```

rule native_pe_header {
  condition:
    uint16(0) == 0x5a4d and
    uint32(uint32(0x3c)) == 0x00004550
}
```

Listing 4: YARA predicate for the native MZ and PE signatures.

PE32 uses a 32-bit optional header and the i686 compiler; PE32+ uses the 64-bit header and the x86-64 compiler. Selected computed

predicates determine which variant is required. Nested relative constants are patched by following the pointer in the generated file rather than assuming a fixed PE-header location.

The ordinary PE path does not patch direct non-structural constants. Such rules require the offset-aware or direct-writer strategy. The router also treats PE-signature bytes extracted at 0x3c as PE evidence without checking the nested flag, which can misroute an unusual direct predicate.

6.4 Offset-Aware PE: Low Alignment and Two Passes

A conventional PE link does not guarantee that a C global will appear at an arbitrary file offset. The offset-aware backend therefore emits a freestanding C program with a dedicated `.oray` section. Listing 5 shows the complete low-alignment link command for PE32+.

```
x86_64-w64-mingw32-gcc \
-o app.exe main.c \
-nostdlib -lkernel32 -e _start \
-Wl,--image-base,0x140000000 \
-Wl,--file-alignment,0x10 \
-Wl,--section-alignment,0x10
```

Listing 5: MinGW command for the offset-aware PE32+ backend.

The custom `_start` calls `GetStdHandle`, `WriteFile`, and `ExitProcess`; no C runtime is required. With equal low file and section alignment, MinGW produces a flat relation in which a section’s raw file pointer and RVA coincide. For raw pointer P , RVA R , and byte index j ,

$$F = P + j, \quad \text{RVA}_{\text{byte}} = R + j, \quad (10)$$

and the intended invariant $P = R$ gives $F = \text{RVA}_{\text{byte}}$.

Because the linker’s section prefix is not predicted, construction uses two passes. The first build places an eight-byte sentinel in `.oray`; searching the probe binary yields its file offset D . A sentinel is a distinctive marker used only to discover placement; Listing 6 shows its source representation. For each requested witness (F_i, B_i) with $F_i \geq D$, the second-pass blob is defined by

```
__attribute__((section(".oray"), used))
unsigned char _oray[] = {
    0xAB, 0x61, 0x72, 0x61, 0x79, 0x53, 0xCD,
    0xEF
};
```

Listing 6: Generated C declaration that places the probe or final blob in `.oray`.

$$\text{blob}[F_i - D : F_i - D + |B_i|] = B_i, \quad (11)$$

with zero-filled gaps. Requests below D are accepted only if the probe’s native headers already contain the required bytes. The program is rebuilt with the final blob, and every exact string interval is read back and compared byte for byte. This verification checks YARA-visible file placement; it does not by itself prove loader behavior or protect against a subsequent overlapping constant patch. MinGW output can also contain build timestamps, so identical inputs and toolchain versions do not imply byte-identical PE files.

6.5 Scanner-Oriented and Generic Writers

The `-scan-only` path bypasses GCC and MinGW. Its ELF writer emits a 64-byte ELF64 header followed by direct byte ranges. Because it emits neither program nor section headers, the result is not runnable. This path also omits nested constants.

The scan-only PE writer emits the native identification chain described in Table 2, then places string witnesses directly. It does not currently receive extracted constants. Moreover, an exact placement at a low offset can overwrite the DOS header, PE signature, COFF header, or Optional Header. The result is intended for raw scanning, not execution or general loader compatibility.

Non-PE magic and low flat layouts use the generic writer. It reserves integer ranges in a zeroed buffer before packing unconstrained strings from offset 0x10. Constants and synthetic nested pointers are then written directly. Known magic bytes determine the output extension, but do not establish that the blob is a valid document or archive. Direct writers use a minimum size of 0x200, so they cannot satisfy smaller filesize upper bounds.

6.6 Filesize Constraints and Acceptance Oracle

Textual filesize comparisons are reduced to a half-open interval $[m, M)$. If the initial artifact size s is below m , a target inside the interval is chosen and bytes are appended. The amount of padding is a deterministic function of (s, m, M) , but the contents use `os.urandom`; padded artifacts are therefore not byte-for-byte reproducible and random bytes can theoretically interact with negative or count-sensitive predicates. If $s \geq M$, the backend warns and cannot shrink the artifact. Parsing is currently regex-based over the rule text and does not reject contradictory bounds, so this mechanism is not a general filesize constraint solver.

Construction completion is not the acceptance criterion. The regular CLI emits an artifact without scanning it; the evaluation driver invokes the external YARA engine with an independently associated upstream source rule. Association requires the source filename, trailing directory path, and first public rule identifier to agree; missing, ambiguous, or name-mismatched associations fail explicitly rather than falling back to the normalized derivative. The selected source rule is reduced to its reachable standalone dependency closure, and the scan is restricted to that identifier so that an unrelated rule cannot produce a false pass. A result is recorded as passed only when this original source predicate matches the generated artifact. This oracle independently tests the concrete synthesized witness against r , while still not proving universal implication between r' and r . Its independence is at the predicate level: the evaluator still shares Aray’s source-selection and dependency-closure machinery and uses the same YARA implementation.

7 Evaluation

7.1 Staged End-to-End Design and Corpus

We ask three sequential questions: how many selected source entries Aray’s normalization workflow admits, how many accepted outputs pass constructibility preflight, and, among those admitted to construction, how often deterministic extraction and realization produce an artifact accepted by the independently

selected upstream original rule? We report *normalization admission*, the overall *positive-fixture yield*, and *conditional realization*, respectively. The two stages are measured separately to attribute model-dependent normalization and deterministic realization, but they form one lineage from public source files to terminal Aray dispositions.

The source corpus consists of 416 files from the public Yara-Rules repository [22] at commit `0f93570194a80d2f2032869055808b0ddcdfb360`. In stage one, `aray-normalize` reduced each file to its selected standalone target and applied Aray’s normalization and validation workflow. With `glm-5.2:cloud` configured for normalization and residual judging, 182 entries followed the deterministic no-normalization path and 234 received model normalization. All 416 outputs were accepted, for 100% normalization admission. Here, *accepted* means that Aray’s deterministic checks and, where needed, its configured judge admitted the candidate; it does not mean human review or a formal proof of equivalence. Before the final freeze, an original-rule scan exposed two anchored-regex counterexamples that a derivative-only oracle had missed. We strengthened the contextual regex validator, repaired those two specializations, and retained their boundary patterns as regression tests. All 416 final original-derivative pairs then returned no error under the supported deterministic syntax and semantic checks. This does not mean that all 416 transformations belonged to the narrow implication fragment that bypasses residual judging. Because this refinement used failures observed in the same corpus, the final result is a post-fix systems evaluation rather than a held-out estimate of normalization generalization.

Those exact outputs were frozen before stage two. The SHA-256 digest of their sorted file-hash manifest is `2b7d09de6a0f152218a471ad018dd50fe9af4a4aae08d5ac7fccfa9cd2cb985d`. Freezing creates an auditable handoff and prevents model-dependent regeneration from obscuring realization failures; it does not make normalization external to Aray. The frozen outputs were evaluated with `scan_only=false` and one worker. The environment provided GCC, both x86-64 and

i686 MinGW, and the YARA CLI. Generic routes remained direct byte writers despite the full-compile setting.

To make accidental inference in stage two observable, both model roles were configured with the name `deterministic-control` and an OpenAI-compatible endpoint at `127.0.0.1:9`, which was expected to be unreachable. Any model invocation would therefore fail the run. A match required successful construction followed by positive output from the YARA CLI against the associated upstream original predicate. The evaluator ran with original-rule validation enabled and the upstream repository as its association root. The final control report records 11.418 seconds as the sum of per-rule durations; this is not a hardware-independent performance benchmark.

7.2 End-to-End Case Study

Listing 7 gives a compact case that exposes each stage without requiring normalization. It is a focused repository example, separate from the aggregate corpus measurement.

```
rule multi_offset {
  strings:
    $a = "alpha"
    $b = "beta"
  condition:
    $a at 0x600 and $b at 0x800
}
```

Listing 7: Two exact-offset strings used for the end-to-end case study.

Source reduction selects `multi_offset`, and the capability classifier takes the no-normalization path. Deterministic extraction produces two ASCII entries with exact file offsets `0x600` and `0x800`; there are no integer entries. With no PE or non-PE magic predicate, routing selects the ELF backend. Code generation emits sections for the two byte ranges, and the linker contract places their VMAs at `ELF_BASE+0x600` and `ELF_BASE+0x800`, making the corresponding file offsets exact.

We reproduced the complete path with an unreachable model endpoint. The output was a statically linked ELF64 executable. Reading five bytes at `0x600` returned `61 6c 70 68 61` (`alpha`), and reading four bytes at `0x800` returned `62 65 74`

Stage and outcome	Rules	Share
<i>Normalization: 416 accepted</i>		
No model normalization	182	43.75%
Model-normalized	234	56.25%
<i>Stage-two terminal disposition</i>		
Upstream original matched	406	97.6%
Expected: unsupported <code>pe.*</code>	7	1.7%
Expected: whole-file hash	2	0.5%
Expected: integer width	1	0.2%
<code>yara_mismatch</code>	0	0.0%
Construction failure	0	0.0%
Total	416	100.0%

Table 3: Sequential stages of the Aray evaluation. Both panels follow the same 416-entry lineage and are not additive. Normalization shares are exact to two decimal places; realization dispositions are rounded to one decimal place. A pass requires a match against the independently associated upstream original rule. Positive-fixture yield was 406/416 and conditional realization was 406/406.

`61 (beta)`. Scanning with YARA reported `multi_offset build/linux/app`; executing the fixture printed the fixed ARAY banner and exited. This case demonstrates the concrete chain from rule predicate, through typed evidence and layout, to a runnable matching artifact. It does not imply that the artifact is the only member, or a preferred member, of $\mathcal{M}_r$.

7.3 Results

The upper panel of Table 3 reports the Aray normalization stage; the lower panel reports terminal dispositions after its frozen outputs entered deterministic realization. Stage one admitted all 416 entries. In stage two, preflight admitted 406, and all 406 produced matching artifacts. The staged positive-fixture yield was therefore 406/416 (97.6%), while conditional realization was 406/406 (100%). The other ten terminated during constructibility preflight: seven used unsupported `pe.*` module semantics, two prescribed whole-file hash values, and one required an integer value outside the declared width. Every admitted artifact matched the independently associated upstream original rule. There were zero `yara_mismatch` outcomes and no construction failures among admitted rules.

Because normalization had already been performed by Aray in stage one, every frozen input

correctly bypassed it in stage two. Both deterministic extractors processed each of the 406 rules that passed preflight; extraction fallback was invoked zero times. The remaining ten did not reach extraction. Thus all 416 frozen inputs reached a terminal disposition without contacting the stage-two control endpoint. Among matches, evaluator output paths identify 258 ELF artifacts, 101 PE artifacts, and 47 generic blobs. The first two groups exercised compiler/linker paths; generic output remained a direct writer. This experiment did not exercise either `-scan-only` writer, and the retained report does not separate ordinary from offset-aware PE.

The original-rule oracle materially affected validation practice. An earlier run produced 404 matches and two `yara_mismatch` outcomes. In `MALW_AlMashreq`, a doubly anchored wide regex had been replaced by a free literal inside a seven-of-nine specialization. In `RAT_PoetRATDoc`, the shortest selected regex branch depended on end-of-file placement. The contextual validator now rejects both proposals: the repaired `AlMashreq` specialization selects seven non-anchored declarations, while `PoetRAT` selects the unanchored `Python` alternative. Both repaired fixtures pass deterministic candidate validation and the final upstream-original oracle. Their boundary patterns are also covered by self-contained regression tests; the complete frozen corpus is not versioned.

7.4 Interpretation

Taken together, the two stages measure workflow completion from Aray’s deterministic source selection and optional normalization through preflight, extraction, realization, and final scanning. This includes toolchain-sensitive ELF and PE realization as well as direct generic writing. The frozen boundary also provides a component result: the deterministic core produced an artifact accepted by the upstream original oracle for every constructible entry in this corpus.

The end-to-end label is operational, not a claim of formally verified semantic equivalence. The upstream-original scan can reveal concrete normalization or realization errors that a derivative-only oracle would miss; it exposed

the two anchored-regex regressions above, and zero such mismatches remained in the final run. This is empirical validation of one synthesized witness per constructible source rule, not a solver-backed proof of universal subset implication or coverage of arbitrary YARA.

The zero-mismatch result after preflight is methodologically important. It shows that known unsupported or impossible cases can be separated from construction errors instead of being reported as undifferentiated failures. The unreachable endpoint is also stronger than inspecting configured model names: it turns an unexpected invocation into an immediate experimental failure.

8 Discussion

8.1 Determinism and Reproducibility

For fixed source and typed input, interpretation, layout planning, and source generation follow deterministic code paths. Routing and post-build patch plans are deterministic as well. This is *logical determinism*: the same constraints lead to the same construction decisions. Byte-for-byte reproducibility is narrower. Compiler and linker versions may change container bytes and section placement, and random filesize padding is explicitly non-reproducible. Exact-offset PE mitigates one toolchain uncertainty by probing and verifying the final file instead of assuming a section origin.

8.2 Safety and Intended Use

Runnable-backend outputs are designed with minimal scaffolding that prints a fixed banner and exits; this evaluation scanned but did not execute or loader-validate every artifact. Scanner-oriented outputs are not executable. Nevertheless, artifacts contain rule-selected bytes and may trigger security controls by design. They should be labeled and handled as detection-test fixtures. ARAY reduces the need to distribute malware, but it does not make every arbitrary YARA witness safe to execute or expose.

Operational use should therefore preserve a containment boundary even when the fixture is designed to be non-malicious. A recovery exercise should use dedicated buckets or prefixes, least-privilege and short-lived identities,

explicit replication scope, denied-by-default egress, centralized audit logs, and an environment that can be rebuilt from a known state after the test. The scanner need only read the fixture; no pipeline component should execute it. Production credentials and unrelated datasets should remain outside the test environment. These controls also bound optional model assistance: Aray exposes no execution tools to the model, but it neither provisions nor validates the surrounding sandbox, cloud account, or recovery topology.

9 Limitations

First, the lexical front end and extractor do not implement a complete YARA AST or a general satisfiability solver. Complex Boolean or arithmetic forms can be unsupported or over-constrained; loops and imported modules are also outside the deterministic fragment. Rule-set reduction chooses one sufficient branch rather than preserving every alternative. Because normalization and fallback triggers are incomplete, parenthesized negation or an ignored modifier can still proceed with incomplete evidence.

Second, witness synthesis is existential rather than one-to-one. A regex, variable hex pattern, disjunction, or unconstrained placement can describe many members of $\mathcal{M}_r$, potentially infinitely many. ARAY returns one candidate selected by stable-order, minimum-length, and feasibility heuristics; it does not prove that this candidate is minimal, canonical, representative of the detected family, or preferable to the alternatives. Conversely, different rules can admit the same fixture. A successful match therefore certifies one positive witness, not an exhaustive realization of the rule.

Third, model normalization is approximate. Deterministic checks establish syntax and rule identity, then verify retained declarations and witness membership, including representative boundary contexts for retained regex witnesses. General subset implication remains judged rather than proven, and an **uncertain** verdict currently proceeds in the regular graph; the batch normalizer used by the evaluation retains only **passed** candidates. The upstream-original oracle independently vali-

dates the concrete artifact generated from each constructible candidate, but does not turn that observation into a universal implication proof. The oracle is independent of the normalized construction predicate, but shares Aray’s source-selection and dependency-closure machinery and the same YARA implementation; its independence is therefore predicate-level, not implementation-level. Extraction fallback validates its schema but has no complete semantic oracle before construction.

Fourth, interval integrity is incomplete across all backends. String placement is collision-checked, but a later constant patch can overlap a string or structural byte. PE read-back occurs before that patch, and constant-constant conflicts are not globally solved. Separately, exact occurrence equality is represented only as an intended lower bound, so incidental bytes can add matches. Integer support also remains incomplete for signed **int16** reads and nested inner functions other than little-endian **uint32**.

Fifth, backend support is uneven. Ordinary runnable PE relies on compiler retention for free globals and does not realize every range or occurrence count; it also skips direct non-structural constants. Scan-only PE forwards no extracted constants, always emits PE32+, and permits low strings to overwrite the header structures listed in Table 2. Scan-only ELF omits nested constants. Generic output is a byte witness rather than a structurally valid document. None of the scanner-oriented outputs should be treated as a runnable format implementation.

Sixth, filesize padding can only enlarge artifacts and uses random data. Tight upper bounds can be impossible for a selected full-compile backend, and random padding can alter count or absence predicates. Textual parsing does not reject contradictory bounds. Toolchain versions and PE timestamps also limit byte-for-byte reproducibility.

Finally, the 406/416 result is the positive-fixture yield of a staged Aray workflow whose normalization phase used GLM-5.2 and whose realization phase used a frozen handoff. It is end to end under an upstream-original-rule oracle: 406 artifacts matched, ten cases ended in expected preflight dispositions, and no con-

structible case produced `yara_mismatch`. The two regex regressions used to strengthen the validator were discovered in this corpus before the final run, so the result is not a held-out estimate. Nor should it be interpreted as formal normalization fidelity, full coverage of unmodified public YARA repositories, or a language-model benchmark. The repository currently versions aggregate provenance and a corpus digest, not the frozen files or canonical per-rule reports, which limits independent reproduction from a clean clone.

10 Related Work

Research on YARA has primarily addressed scalable intelligence and detector generation. YARIX organizes rule-based malware intelligence at scale [3], but does not construct positive objects for testing an ingestion path. AutoYARA infers rules from samples through bi-clustering [16]; PackGenome generates packer signatures [11]; NeuroYara ranks generated rules [12]; and APIARY derives rules from behavioral API data [5]. Living off the Analyst extracts rule features for a separate classifier [8]. Their principal direction is from samples, behavior, or existing rules toward a detector. ARAY reverses that direction: it treats an existing rule as the specification, constructs one positive file-level witness, and preserves the construction predicate for inspection. In the reported evaluation, the evaluator additionally uses the derivative-independent upstream original rule as its acceptance oracle.

Recent work has used language models to extract security knowledge or generate detection rules from reports and packages [4, 9, 17, 24]. Those systems place learned generation on the detector-construction path. ARAY instead uses a model only behind an explicit normalization/fallback boundary and keeps supported extraction, source templates, layout, and serialization under conventional code. Binary realization is governed by exact offsets, integer widths, alignments, and container invariants; encoding these operations as code makes them testable and permits read-back verification. Model-selected literals remain untrusted payload input. This follows the program-synthesis principle of separating a specification from a constrained realizer [7], while avoiding uncon-

strained generation at the byte boundary.

LLM-assisted cyber evaluation presents a related but distinct containment problem. ExploitGym packages realistic userspace, browser, and kernel targets in controlled environments and reports that Claude Mythos Preview produced working exploits for 157 of 898 instances; enabling standard mitigations reduced, but did not eliminate, model success [21]. Anthropic’s agentic misalignment study provides a different class of evidence: models with tool access leaked information or took other harmful actions in deliberately constrained simulations, while the authors explicitly report no corresponding behavior observed in real deployments [1]. Kimi K3 has no comparable public containment incident; its primary documentation instead reports long-horizon terminal orchestration and warns that excessive proactiveness can produce unexpected decisions on a user’s behalf [14]. These results are therefore not interchangeable with the observed OpenAI–Hugging Face intrusion. Together, however, they motivate least privilege, independent action mediation, restricted egress, ephemeral credentials, monitoring, and rebuildable environments whenever an LLM is given tools in a security or recovery exercise. Aray adopts the narrower architectural response of treating model output as untrusted declarative input and reserving construction authority for deterministic code.

The work is also related to constrained input generation and program synthesis. G2Fuzz synthesizes generators for non-textual fuzzing inputs [23], whereas ARAY does not seek broad input diversity or fuzzing coverage. It realizes one existential witness for a selected YARA predicate, including exact file offsets and executable-container constraints, while the reported evaluation checks the result with the external YARA engine against the associated upstream original predicate. Thus its novelty lies in turning an operational detector into an auditable, format-aware positive fixture, not in general-purpose file generation.

11 Future Work

The highest-priority improvement is a grammar-backed condition AST coupled to a joint constraint solver. The solver should

reason about content, placement, multiplicity, absence, and filesize without assigning each evidence class in isolation. A unified interval plan can then reserve strings and constants before construction and verify them again after patching and padding. Deterministic witness generation and contextual validation should also cover more regex and nonlinear hex forms, while a formal or solver-backed subset check further narrows the model boundary. Until that implication check is available, a strict mode should stop normalization on an **uncertain** verdict rather than admitting the candidate to construction.

The medium-term priority is module-aware synthesis. Although ARAY generates PE containers, it does not currently interpret conditions from `import "pe"`. Initial support should translate selected structural fields, section properties, and import/export predicates into typed PE constraints. A field-level capability matrix should distinguish predicates that are constructible, merely verifiable, infeasible, or not yet supported. The same architecture can later target file-derived modules such as `elf`, `dotnet`, and `lnk`. Modules that depend on external observations require a separate metadata contract, while functions involving prescribed hashes must retain explicit infeasibility checks.

Backend verification should parse final ELF program headers and PE section tables rather than rely only on construction-time assumptions. It should also forward supported constants to scan-only PE and produce structurally valid generic containers where format validity matters. Versioned toolchain environments and a deterministic, predicate-aware padding mode would improve reproducibility.

In the longer term, ARAY should support YARA-X, which is intended as YARA’s successor but is not API-compatible with YARA 4.x [20]. A scanner abstraction should accommodate both engines without assuming that rule-level compatibility implies identical semantics. Differential tests should cover syntax differences, module outputs, and match results. Reporting acceptance against both engines would make compatibility an observed property rather than an assumption.

Finally, future evaluations should retain the

upstream-original oracle while also reporting a derivative-rule scan. The pair helps attribute failures: a derivative-only match is evidence of a normalization-fidelity problem, while failure of the derivative indicates an extraction or realization problem. The four possible outcomes should be reported explicitly rather than treated as a complete binary diagnosis. Complete per-rule reports, source-rule associations, artifact hashes, tool versions, and normalization-validation provenance should be published. Replication across held-out repositories, commits, and YARA engines would provide a stronger estimate of external validity than the post-fix corpus reported here. Table-generation scripts should derive every aggregate result, and boundary failures should remain separately attributable.

12 Conclusion

ARRAY reframes positive YARA validation as selecting and synthesizing one file-level witness from a rule’s match set. The current system is best understood as a deterministic interpreter and binary realizer with optional model-assisted normalization, not as an LLM binary generator. Conventional code derives the supported string and integer evidence and assigns its file layout. The ELF backend enforces an explicit mapping between file offsets and virtual addresses. The offset-aware PE backend instead discovers linker placement in a probe build and verifies requested bytes after rebuilding.

In the first stage of the evaluation, Aray’s own normalization workflow accepted all 416 selected public-rule entries: 182 (43.75%) required no model normalization, while 234 (56.25%) were model-normalized. All final original-derivative pairs returned no error under the supported deterministic checks before the frozen handoff; this is distinct from proving every pair in the narrow implication fragment. Preflight admitted 406, and all 406 artifacts matched their independently associated upstream original rules, yielding 406/416 (97.6%) end-to-end positive-fixtured yield, 406/406 (100%) conditional realization, and zero `yara_mismatch` outcomes. The other ten received expected terminal dispositions, and no model endpoint was contacted during

the second stage. This independently validates the concrete witnesses without claiming universal implication or general coverage of YARA. The results demonstrate that a substantial and auditable deterministic core can turn normalized YARA predicates into portable test fixtures without distributing the corresponding malware. Such fixtures can exercise object-ingestion and recovery paths without maintaining a matching live-malware corpus in every test environment, reducing—but not eliminating—the need for isolated, monitored, and resettable infrastructure.

13 Availability

ARAY is open source under the Apache License 2.0 at

<https://github.com/c2dc/array>

The repository includes the interpretation pipeline and all binary backends, together with their CLI and tests. It also versions diagram sources and a machine-readable summary of both stages of the 416-entry evaluation, including the original-rule oracle configuration and aggregate dispositions. The summary is stored under `docs/site/assets/as-yara-rules-416-deterministic.json`.

References

- [1] Anthropic. Agentic misalignment: How LLMs could be insider threats. [Anthropic research report](#), 2025. Published June 20, 2025; accessed August 16, 2026.
- [2] Michael Bartock, Jeffrey Cichonski, Murugiah Souppaya, Matthew Smith, Gregory Witte, and Karen Scarfone. Guide for cybersecurity event recovery. Technical Report NIST Special Publication 800-184, National Institute of Standards and Technology, 2016.
- [3] Michael Brengel and Christian Rossow. YARIX: Scalable YARA-based malware intelligence. In *30th USENIX Security Symposium (USENIX Security 21)*, pages 3541–3558, 2021.
- [4] Marvin Büchel, Tommaso Paladini, Stefano Longari, Michele Carminati, Stefano Zanero, Hodaya Binyamini, Gal Engelberg, Dan Klein, Giancarlo Guizzardi, Marco Caselli, Andrea Continella, Maarten van Steen, Andreas Peter, and Thijs van Ede. SoK: Automated TTP extraction from CTI reports – are we there yet? In *34th USENIX Security Symposium (USENIX Security 25)*, pages 4621–4641, 2025.
- [5] Antonio Coscia, Roberto Lorusso, Antonio Maci, and Giuseppe Urbano. APIARY: An api-based automatic rule generator for YARA to enhance malware detection. *Computers & Security*, 153: 104397, 2025. doi: 10.1016/j.cose.2025.104397.
- [6] Free Software Foundation. *GNU ld: The GNU Linker*. GNU Project, 2026. Accessed August 15, 2026.
- [7] Sumit Gulwani, Oleksandr Polozov, and Rishabh Singh. Program synthesis. *Foundations and Trends in Programming Languages*, 4(1–2):1–119, 2017. doi: 10.1561/25000000010.
- [8] Siddhant Gupta, Fred Lu, Andrew Barlow, Edward Raff, Francis Ferraro, Cynthia Matuszek, Charles Nicholas, and James Holt. Living off the analyst: Harvesting features from YARA rules for malware detection. *arXiv preprint arXiv:2411.18516*, 2024. doi: 10.48550/arXiv.2411.18516.
- [9] Xiaowei Hu, Haoning Chen, Huaifeng Bao, Wen Wang, Feng Liu, Guoqiao Zhou, and Peng Yin. A LLM-based agent for the automatic generation and generalization of IDS rules. In *2024 IEEE 23rd International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*, pages 1875–1880, 2024. doi: 10.1109/TrustCom63139.2024.00259.
- [10] Hugging Face. Anatomy of a frontier lab agent intrusion: A technical timeline of the july 2026 incident. [Hugging Face technical timeline](#), 2026. Published July 27, 2026; accessed August 16, 2026.
- [11] Shijia Li, Jiang Ming, Pengda Qiu, Qiyuan Chen, Lanqing Liu, Huaifeng Bao, Qiang Wang, and Chunfu Jia. Packgenome: Automatically generating robust YARA rules for accurate malware packer detection. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, pages 3078–3092, 2023. doi: 10.1145/3576915.3616625.
- [12] Ziad Mansour, Weihang Ou, Steven H. H. Ding, Mohammad Zulkernine, and Philippe Charland. Neuroyara: Learning to rank for YARA rules generation through deep language modeling and discriminative n-gram encoding. *IEEE Transactions on Dependable and Secure Computing*, 22(2):1747–1762, 2025. doi: 10.1109/TDSC.2024.3449641.
- [13] Microsoft. PE format: Portable executable and common object file format specification. <https://learn.microsoft.com/en-us/windows/win32/debug/pe-format>, 2026. Accessed August 15, 2026.
- [14] Moonshot AI. Kimi K3: Open frontier intelligence. [Kimi technical blog](#), 2026. Published July 2026; accessed August 16, 2026.

- [15] OpenAI. OpenAI and Hugging Face partner to address security incident during model evaluation. [OpenAI incident report](#), 2026. Published July 21, 2026; accessed August 16, 2026.
- [16] Edward Raff, Richard Zak, Gary Lopez Munoz, William Fleming, Hyrum S. Anderson, Bobby Filar, Charles Nicholas, and James Holt. Automatic YARA rule generation using biclustering. In *Proceedings of the 13th ACM Workshop on Artificial Intelligence and Security (AISec)*, pages 71–82, 2020. doi: 10.1145/3411508.3421372.
- [17] Yuval Schwartz, Lavi Ben-Shimol, Dudu Mirman, Yuval Elovici, and Asaf Shabtai. LLMCloudHunter: Harnessing LLMs for automated extraction of detection rules from cloud-based CTI. In *Proceedings of the ACM Web Conference 2025*, pages 1922–1941, 2025. doi: 10.1145/3696410.3714798.
- [18] System V ABI Project. System v application binary interface: Amd64 architecture processor supplement. <https://gitlab.com/x86-psABIs/x86-64-ABI>, 2026. Accessed August 15, 2026.
- [19] VirusTotal. YARA documentation. <https://yara.readthedocs.io/>, 2026. Accessed August 15, 2026.
- [20] VirusTotal. YARA-X documentation. <https://virustotal.github.io/yara-x/>, 2026. Accessed August 16, 2026.
- [21] Zhun Wang, Nico Schiller, Hongwei Li, Srijiith Seshan Narayana, Milad Nasr, Nicholas Carlini, Xiangyu Qi, Eric Wallace, Elie Bursztein, Luca Invernizzi, Kurt Thomas, Yan Shoshitaishvili, Wenbo Guo, Jingxuan He, Thorsten Holz, and Dawn Song. ExploitGym: Can AI agents turn security vulnerabilities into real attacks? *arXiv preprint arXiv:2605.11086*, 2026. doi: 10.48550/arXiv.2605.11086.
- [22] Yara-Rules Project. Yara-rules: A collection of YARA rules. <https://github.com/Yara-Rules/rules>, 2026. Corpus commit 0f93570194a80d2f2032869055808b0ddcdcfb360.
- [23] Kunpeng Zhang, Zongjie Li, Daoyuan Wu, Shuai Wang, and Xin Xia. Low-cost and comprehensive non-textual input fuzzing with LLM-synthesized input generators. In *34th USENIX Security Symposium (USENIX Security 25)*, pages 6999–7018, 2025.
- [24] XiangRui Zhang, XueJie Du, HaoYu Chen, Yongzhong He, Wenjia Niu, and Qiang Li. Automatically generating rules of malicious software packages via large language model. In *2025 55th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, pages 734–747, 2025. doi: 10.1109/DSN64029.2025.00072.